

Kirana Connect

Technology Stack & Render Deployment Guide

Comprehensive Documentation for Developers

1. Project Structure Overview

Kirana Connect is a hyperlocal grocery marketplace with 3 main components:

```
kirana-app/  
% % % backend/           !' Node.js REST API + Socket.io (Real-time)  
% % % customer-app/      !' React Native Expo (Customer mobile app)  
% % % store-app/         !' React Native Expo (Store owner mobile app)
```

2. Backend — Package Documentation

The backend is a Node.js + Express REST API server that manages all business logic, database interactions, authentication, and real-time communication.

2.1 Core Framework

express v4.18.2

& Web Framework

What it does: Fast, minimalist web framework for Node.js. Handles HTTP routing, middleware, and request/response lifecycle.

Why we need it: To define API endpoints like GET /stores, POST /auth/login, and structure the backend with middleware layers (auth, rate limiting, logging).

```
const express = require('express');  
const app = express();  
app.get('/stores', async (req, res) => {  
  const stores = await db.query('SELECT * FROM stores');  
  res.json(stores.rows);  
});  
app.listen(process.env.PORT || 4000);
```

cors v2.8.5

& Security / Middleware

Enables Cross-Origin Resource Sharing so the mobile apps (running on different ports/domains) can call the backend API securely.

```
app.use(cors({ origin: process.env.ALLOWED_ORIGINS.split(',') }));
```

helmet v7.1.0

& Security

Sets secure HTTP headers automatically to protect against common web vulnerabilities like XSS and clickjacking.

```
app.use(helmet());
```

morgan v1.10.0

& Logging

HTTP request logger middleware. Logs every incoming request to the console for debugging and monitoring.

```
app.use(morgan('dev')); // Output: GET /stores 200 12ms
```

express-rate-limit v7.1.5

& Security

Limits repeated requests to APIs to prevent abuse and DDoS attacks (e.g., max 100 requests per 15 minutes per IP).

2.2 Database

pg (node-postgres) v8.11.3

& Database Client

What it does: Official PostgreSQL client for Node.js. Connects to the PostgreSQL database and executes SQL queries.

Why we need it: All app data (users, stores, orders, products) is stored in PostgreSQL. The pg library is how the server reads and writes this data.

```
const { Pool } = require('pg');
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false,
});
const result = await pool.query('SELECT * FROM users WHERE id = $1', [userId]);
```

2.3 Real-time Communication

socket.io v4.7.2

& Real-time / WebSockets

What it does: Enables bidirectional, event-based real-time communication between server and clients.

Why we need it: When a customer places an order, the store owner needs to be instantly notified — without refreshing the screen. Socket.io pushes live updates for order status changes.

```
// Server: emit when order is placed
io.to(`store:${storeId}`).emit('new_order', orderData);

// Client (Store App): listen for new orders
socket.on('new_order', (order) => { showNotification(order); });
```

2.4 Authentication & Security

jsonwebtoken (JWT) v9.0.2

& Authentication

Generates and verifies JSON Web Tokens for stateless user authentication. After login, users receive a token they send with every API request.

```
// Generate token on login
const token = jwt.sign({ userId: user.id, role: user.role }, process.env.JWT_SECRET,
{ expiresIn: '30d' });

// Verify token on protected routes
jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
  req.user = decoded;
});
```

bcryptjs v2.4.3

& Password Security

Hashes passwords using the bcrypt algorithm. Passwords are never stored in plain text — only the hash is saved, making the app secure even if the database is exposed.

```
// On registration
const hashedPassword = await bcrypt.hash(plainPassword, 10);

// On login
const isValid = await bcrypt.compare(plainPassword, hashedPassword);
```

2.5 File Uploads & Image Management

multer v1.4.5

& File Upload

Middleware for handling multipart/form-data, used for uploading product images from the store owner app.

```
const upload = multer({ storage: multer.memoryStorage() });
app.post('/items/upload', upload.single('image'), handler);
```

cloudinary v1.41.0

& Cloud Storage

Cloud-based image storage. After multer captures the image, we upload it to Cloudinary and store the public URL in the database. This avoids storing images on the server disk.

```
cloudinary.uploader.upload_stream({ folder: 'kirana-products' }, (err, result) => {
  const imageUrl = result.secure_url; // save this URL in PostgreSQL
});
```

2.6 Validation & Utilities

zod v3.22.4

& Validation

Schema-based validation for incoming API request data. Ensures all required fields exist and are of the correct type before processing.

```
const orderSchema = z.object({
  storeId: z.string().uuid(),
  items: z.array(z.object({ itemId: z.string(), qty: z.number().min(1) })))
});
const validated = orderSchema.parse(req.body);
```

dotenv v16.3.1

& Configuration

Loads environment variables from a .env file into process.env. This keeps secrets like DB passwords and API keys out of the code.

```
require('dotenv').config();
const dbUrl = process.env.DATABASE_URL;
```

uuid v9.0.0

& Utility

Generates universally unique IDs used for order IDs and other records where sequential integers are not suitable.

```
const { v4: uuidv4 } = require('uuid');
```

```
const orderId = uuidv4(); // e.g., '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d'
```

pdftkit v0.14.0

& Report Generation

Generates PDF reports (like invoices and sales summaries) programmatically. Used in the /reports route to produce downloadable PDFs for store owners.

fast-csv v4.3.6

& Data Export

Parses and generates CSV files. Used to export order data or inventory lists as downloadable spreadsheets.

3. Frontend Apps — Package Documentation

Both Customer App and Store App are built with React Native + Expo, sharing the same core packages but differing in features.

3.1 Core Framework

react v19.1.0 + react-native v0.81.5

& **UI Framework**

React is the JavaScript library for building UI. React Native compiles React code into native Android and iOS components, providing true native performance.

expo ~54.0.33

& **Mobile Platform**

A framework on top of React Native that provides pre-built native modules, OTA updates, and a build system. Massively reduces the complexity of building mobile apps.

```
npx expo start          # Start dev server
npx expo start --android # Open on Android emulator
npx expo start --web     # Open in browser
```

expo-router ~6.0.23

& **Navigation / Routing**

File-based routing — any file created in the app/ folder automatically becomes a screen. No manual route configuration needed.

```
app/
% % % (tabs)/
%   % % % index.tsx      !' "/" tab screen
%   % % % orders.tsx     !' "/orders" tab screen
%   % % % profile.tsx    !' "/profile" tab screen
% % % store/[id].tsx     !' "/store/123" dynamic screen
```

3.2 Maps & Location

react-native-maps v1.27.2 + expo-location ~19.0.8

& **Maps / GPS**

expo-location gets the device's GPS coordinates. react-native-maps renders a fully interactive MapView with markers to show nearby stores.

```
import * as Location from 'expo-location';
import MapView, { Marker } from 'react-native-maps';

const { coords } = await Location.getCurrentPositionAsync({});
// coords.latitude, coords.longitude

<MapView initialRegion={{ latitude: coords.latitude, longitude: coords.longitude,
latitudeDelta: 0.05, longitudeDelta: 0.05 }}>
  <Marker coordinate={{ latitude: 12.9716, longitude: 77.5946 }} title="Store Name" />
</MapView>
```

3.3 Networking

axios v1.16.0

& HTTP Client

Promise-based HTTP client used to make API calls to the backend. Supports interceptors for auto-attaching auth tokens to every request.

```
import axios from 'axios';

const api = axios.create({ baseURL: 'http://localhost:4000' });

// Auto-attach token to every request
api.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${token}`;
  return config;
});

const stores = await api.get('/stores');
```

socket.io-client v4.8.3

& Real-time / WebSockets

The client-side counterpart to the server's socket.io. Connects the app to the backend for real-time order updates.

```
import { io } from 'socket.io-client';

const socket = io('http://localhost:4000', { auth: { token: userToken } });
socket.emit('join_store', storeId);
socket.on('new_order', (order) => { alert('New order received!'); });
```

3.4 State Management

zustand v5.0.12

& Global State

Lightweight state management. Replaces Redux for sharing global state (like cart items, user auth data) across all screens without complex boilerplate.

```
import { create } from 'zustand';

const useCartStore = create((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  clearCart: () => set({ items: [] }),
}));

// In any screen:
const { items, addItem } = useCartStore();
```

3.5 UI & Animations

react-native-reanimated ~4.1.1

& Animations

High-performance animations running on the native thread. Used for smooth transitions, swipe gestures, and micro-animations in the UI.

react-native-gesture-handler ~2.28.0

& Gestures

Handles touch gestures like swipe-to-delete, drag & drop, and pinch-to-zoom natively with proper

performance.

@expo/vector-icons v15.0.3

& Icons

Provides access to thousands of icons from popular sets like Ionicons, MaterialIcons, and FontAwesome.

```
import { Ionicons } from '@expo/vector-icons';  
<Ionicons name="cart-outline" size={24} color="#4f46e5" />
```


4. Render Deployment — Step by Step

Step 1: Push Code to GitHub

Render connects to GitHub to auto-deploy your code. If not already done:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/YOUR_USERNAME/kirana-connect.git
git push -u origin main
```

& Make sure `.env` is in your `.gitignore` — never push secret keys to GitHub!

Step 2: Create PostgreSQL on Render

1. Go to <https://render.com/dashboard>
2. Click "New +" ! "PostgreSQL"
3. Name: kirana-connect-db | Region: Singapore | Plan: Free
4. Click "Create Database"
5. Copy the "Internal Database URL" for Step 3.

Step 3: Create Web Service on Render

1. Go to <https://render.com/dashboard>
2. Click "New +" ! "Web Service"
3. Connect GitHub ! Select your repository

Field	Value
Name	kirana-connect-backend
Region	Singapore (same as DB)
Root Directory	backend
Runtime	Node
Build Command	npm install
Start Command	npm start
Plan	Free

Step 4: Set Environment Variables on Render

In the "Environment" tab of your Web Service, add these key-value pairs:

Key	Value
NODE_ENV	production
DATABASE_URL	(paste Internal DB URL from Step 2)
JWT_SECRET	kirana_super_secret_change_this_123
MOCK_DB	false
CLOUDINARY_CLOUD_NAME	(from Cloudinary dashboard)
CLOUDINARY_API_KEY	(from Cloudinary dashboard)
CLOUDINARY_API_SECRET	(from Cloudinary dashboard)
ALLOWED_ORIGINS	*

& Setting `MOCK_DB=false` is critical — without it, data will not be saved to the real database.

Step 5: Run Database Migrations

After the service is live, set up the database tables by running from your local machine:

```
# Windows PowerShell
$env:DATABASE_URL="your_render_db_url"
$env:NODE_ENV="production"
node src/db/migrate.js
node src/db/seed.js
```

Step 6: Update Frontend API URLs

After deployment, Render gives you a URL like <https://kirana-connect-backend.onrender.com>

Find and update the API base URL in both apps:

```
// customer-app/utils/api.js & store-app/utils/api.js
const API_URL = 'https://kirana-connect-backend.onrender.com';

// socket.js
const socket = io('https://kirana-connect-backend.onrender.com', { auth: { token } });
```

& Pro Tip: Use `EXPO_PUBLIC_API_URL` env variable in `.env` files to switch between dev and production easily.

Step 7: Verify Deployment

Test these endpoints to confirm the backend is running:

```
GET https://kirana-connect-backend.onrender.com/stores
POST https://kirana-connect-backend.onrender.com/auth/login
GET https://kirana-connect-backend.onrender.com/catalog
```

& p Free Tier Limitations

Issue	Explanation
Spin-down	Service sleeps after 15 min of inactivity. First request takes ~30s to wake up.
DB Storage	Free PostgreSQL limit is 256MB.
No Persistent Disk	Never store files locally on server disk. Always use Cloudinary for images.

Ø=Ý Auto-Deploy (CI/CD)

Once connected, every git push to your main branch automatically triggers a new Render deployment. No manual steps required!

Deployment Flow:
git push !' GitHub !' Render detects change !' npm install !' npm start !' Live!